

STAT452 Comprehensive R Syntax Reference

Multiple Linear Regression, Polynomial Regression, Regularization, Variable Selection, Logistic Regression, and Classification

STAT452 – Applied Statistics for Engineers and Scientists II

Contents

1	How to use this reference	1
2	Basic R syntax	1
2.1	Values, assignment, and help	1
2.2	Vectors, matrices, and data frames	1
2.3	Data input and preparation	2
3	Graphics and exploratory analysis	2
4	Topic 1: Multiple linear regression	3
4.1	Model formulas	3
4.2	Fit, summarize, and inspect	3
4.3	Coefficients and fitted values	3
4.4	Manual simple-regression calculations	4
4.5	SSE, MSE, and R^2	4
4.6	Matrix least squares	4
4.7	Coefficient tests and confidence intervals	5
4.8	Prediction	5
4.9	Nested models and partial F-tests	5
4.10	Diagnostics and influence	5
5	Topic 2: Polynomial regression and transformations	6
5.1	Polynomial terms	6
5.2	Orthogonal polynomials	6
5.3	Centering and stationary points	6
5.4	Polynomial predictions	6
5.5	Transformations	7
6	Topic 3: Ridge, LASSO, and variable selection	7
6.1	OLS baseline and correlation	7
6.2	Ridge regression with MASS	7
6.3	Ridge and LASSO with glmnet	8
6.4	Soft thresholding	8
6.5	AIC, BIC, and stepwise selection	8
6.6	Cross-validation	9
6.7	Leave-one-out cross-validation	9

7	Topic 4: Logistic regression and classification	9
7.1	Create a binary response	9
7.2	Fit a logistic model	10
7.3	Odds ratios	10
7.4	Predicted probabilities	10
7.5	Classification	11
7.6	Confusion-matrix metrics	11
7.7	Compare thresholds	11
7.8	Deviance, AIC, and nested logistic models	12
7.9	Logistic diagnostics	12
8	Training/test validation	12
9	Common mistakes and final checklist	13

1 How to use this reference

This is a practical syntax reference for the four course topics. The examples use built-in `mtcars` data whenever possible. Run commands one section at a time in R or RStudio. Lines beginning with `#` are comments.

2 Basic R syntax

2.1 Values, assignment, and help

```
# This is a comment
x <- 5
x == 5
c(1, 2, 3)
seq(0, 10, by = 2)
rep(1:3, times = 2)
length(x)
class(x)
typeof(x)
is.na(x)
?mean
help(lm)
getwd()
setwd("path/to/folder")
set.seed(452)
```

Use `<-` for assignment and `==` for an equality test. A function is called with parentheses, for example `mean(x)`.

2.2 Vectors, matrices, and data frames

```
v <- c(2, 4, 6, 8)
v[1]
v[2:4]
v[-1]
v[v > 4]
```

```
M <- matrix(1:6, nrow = 2, byrow = TRUE)
M[1, 2]
t(M)
dim(M)
nrow(M)
ncol(M)

d <- data.frame(
  y = c(10, 12, 15),
  x = c(1, 2, 3),
  group = factor(c("A", "A", "B"))
)
d$y
d[, c("y", "x")]
d[1:2, ]
subset(d, x > 1)
d$new_variable <- d$x^2
names(d)
str(d)
head(d)
summary(d)
```

2.3 Data input and preparation

```
data(mtcars)
cars <- mtcars

d <- read.csv("data.csv")
write.csv(cars, "cars.csv", row.names = FALSE)

na.omit(d)
complete.cases(d)
d[complete.cases(d), ]

factor(c("No", "Yes"), levels = c("No", "Yes"))
factor(x, labels = c("automatic", "manual"))
droplevels(d)

mean(x)
mean(x, na.rm = TRUE)
median(x, na.rm = TRUE)
sd(x, na.rm = TRUE)
var(x, na.rm = TRUE)
min(x)
max(x)
range(x)
quantile(x, probs = c(.25, .5, .75), na.rm = TRUE)
sum(x)
table(d$group)
prop.table(table(d$group))
scale(x)
```

3 Graphics and exploratory analysis

```

plot(x, y)
plot(y ~ x, data = d)
hist(x, breaks = 10)
boxplot(y ~ group, data = d)
pairs(d[, c("y", "x1", "x2")])
cor(d[, c("x1", "x2", "x3")], use = "complete.obs")

plot(x, y, pch = 19, col = "navy",
     main = "Response versus predictor",
     xlab = "Predictor", ylab = "Response")
abline(lm(y ~ x, data = d), col = "red", lwd = 2)

par(mfrow = c(2, 2))
plot(fit)
par(mfrow = c(1, 1))

```

4 Topic 1: Multiple linear regression

4.1 Model formulas

```

y ~ x
y ~ x1 + x2
y ~ .
y ~ x - 1
y ~ 0 + x
y ~ x1 * x2
y ~ x1:x2
y ~ x + factor(group)

```

The formula `y ~ x1 + x2` means an intercept plus two predictors. The formula `x1 * x2` includes both main effects and their interaction. The formula `x1:x2` includes the interaction only.

4.2 Fit, summarize, and inspect

```

fit <- lm(mpg ~ wt + hp, data = mtcars)
fit
summary(fit)
coef(fit)
coefficients(fit)
confint(fit)
vcov(fit)
formula(fit)
model.frame(fit)
model.matrix(fit)

```

4.3 Coefficients and fitted values

```

coef(fit)["wt"]
coef(summary(fit))
summary(fit)$coefficients
summary(fit)$sigma
summary(fit)$r.squared

```

```
summary(fit)$adj.r.squared
df.residual(fit)

y_hat <- fitted(fit)
e <- resid(fit)
head(data.frame(
  observed = mtcars$mpg,
  fitted = y_hat,
  residual = e
))
max(abs(mtcars$mpg - (y_hat + e)))
sum(e)
```

For a multiple-regression coefficient, always state that the other predictors are held fixed.

4.4 Manual simple-regression calculations

```
x <- mtcars$wt
y <- mtcars$mpg
x_bar <- mean(x)
y_bar <- mean(y)

Sxx <- sum((x - x_bar)^2)
Sxy <- sum((x - x_bar) * (y - y_bar))
b1 <- Sxy / Sxx
b0 <- y_bar - b1 * x_bar
c(b0 = b0, b1 = b1)
```

4.5 SSE, MSE, and R^2

```
SSE <- sum(e^2)
SST <- sum((y - mean(y))^2)
SSR <- SST - SSE
MSE <- SSE / df.residual(fit)
RMSE <- sqrt(MSE)
R2 <- 1 - SSE / SST

c(SSE = SSE, SST = SST, SSR = SSR,
  MSE = MSE, RMSE = RMSE, R2 = R2)
```

4.6 Matrix least squares

The least-squares estimator is

$$\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}.$$

```
X <- model.matrix(fit)
Y <- matrix(mtcars$mpg, ncol = 1)

t(X)
t(X) %*% X
solve(t(X) %*% X)
beta_hat <- solve(t(X) %*% X, t(X) %*% Y)
beta_hat
coef(fit)
```

```
drop(t(X) %*% matrix(resid(fit), ncol = 1))
```

Use `%*%` for matrix multiplication. Use `*` for element-by-element multiplication.

4.7 Coefficient tests and confidence intervals

```
tab <- coef(summary(fit))
estimate <- tab["hp", "Estimate"]
standard_error <- tab["hp", "Std. Error"]
t_value <- estimate / standard_error
df_error <- df.residual(fit)
p_value <- 2 * pt(abs(t_value),
                  df = df_error,
                  lower.tail = FALSE)
c(t = t_value, df = df_error, p = p_value)
confint(fit, "hp", level = 0.95)
```

4.8 Prediction

```
new_car <- data.frame(wt = 3, hp = 110)

predict(fit, newdata = new_car)
predict(fit, newdata = new_car,
        interval = "confidence", level = 0.95)
predict(fit, newdata = new_car,
        interval = "prediction", level = 0.95)
```

A confidence interval estimates the mean response. A prediction interval estimates one new individual response and is wider.

4.9 Nested models and partial F-tests

```
reduced <- lm(mpg ~ wt, data = mtcars)
full <- lm(mpg ~ wt + hp, data = mtcars)

anova(reduced, full)
anova(full)
drop1(full, test = "F")
```

4.10 Diagnostics and influence

```
par(mfrow = c(2, 2))
plot(fit)
par(mfrow = c(1, 1))

rstandard(fit)
rstudent(fit)
hatvalues(fit)
cooks.distance(fit)
influence.measures(fit)
dfbetas(fit)
```

```
which.max(abs(rstandard(fit)))
which.max(cooks.distance(fit))
```

The four standard diagnostic plots check linearity and spread, normality, constant variance, and leverage/influence.

5 Topic 2: Polynomial regression and transformations

5.1 Polynomial terms

```
linear <- lm(mpg ~ wt, data = mtcars)
quadratic <- lm(mpg ~ wt + I(wt^2), data = mtcars)
cubic <- lm(mpg ~ wt + I(wt^2) + I(wt^3), data = mtcars)

summary(quadratic)
anova(linear, quadratic, cubic)
AIC(linear, quadratic, cubic)
BIC(linear, quadratic, cubic)
```

Use `I(wt^2)` to make R evaluate the arithmetic expression inside a formula. Retain lower-order terms when higher-order terms are included.

5.2 Orthogonal polynomials

```
orthogonal_fit <- lm(mpg ~ poly(wt, degree = 2), data = mtcars)
raw_fit <- lm(mpg ~ poly(wt, degree = 2, raw = TRUE),
             data = mtcars)
summary(orthogonal_fit)
summary(raw_fit)
```

`poly(wt, 2)` uses an orthogonal polynomial basis by default. `raw = TRUE` gives ordinary powers of `wt`.

5.3 Centering and stationary points

```
mtcars$wt_c <- mtcars$wt - mean(mtcars$wt)
centered_fit <- lm(mpg ~ wt_c + I(wt_c^2), data = mtcars)

b <- coef(centered_fit)
turning_point_centered <-
  -b["wt_c"] / (2 * b["I(wt_c^2)"])
turning_point_original <-
  turning_point_centered + mean(mtcars$wt)
```

5.4 Polynomial predictions

```
grid <- data.frame(
  wt = seq(min(mtcars$wt),
           max(mtcars$wt),
           length.out = 100)
)
grid$fit <- predict(quadratic, newdata = grid)

plot(mpg ~ wt, data = mtcars, pch = 19)
```

```
lines(grid$wt, grid$fit, col = "blue", lwd = 2)

predict(quadratic, newdata = data.frame(wt = c(1, 3, 6)))
range(mtcars$wt)
```

Predictions outside the observed range are extrapolations and may be highly unstable.

5.5 Transformations

```
log_y_fit <- lm(log(mpg) ~ wt, data = mtcars)
log_x_fit <- lm(mpg ~ log(wt), data = mtcars)
log_log_fit <- lm(log(mpg) ~ log(wt), data = mtcars)

sqrt_fit <- lm(sqrt(mpg) ~ wt, data = mtcars)
inverse_fit <- lm(mpg ~ I(1 / wt), data = mtcars)
power_fit <- lm(mpg ~ I(wt^0.5), data = mtcars)

summary(log_log_fit)
par(mfrow = c(2, 2))
plot(log_log_fit)
par(mfrow = c(1, 1))
```

Common transformations are $\log(x)$, $\sqrt{x}$, x^2 , $1/x$, and $\exp(x)$. Check residuals after transforming.

6 Topic 3: Ridge, LASSO, and variable selection

6.1 OLS baseline and correlation

```
full <- lm(mpg ~ wt + hp + qsec + disp + drat + gear,
          data = mtcars)
summary(full)

predictors <- mtcars[, c("wt", "hp", "qsec",
                        "disp", "drat", "gear")]
cor(predictors)
X_standardized <- scale(predictors)
```

6.2 Ridge regression with MASS

```
library(MASS)

lambda_grid <- seq(0, 30, length.out = 100)
ridge_fit <- lm.ridge(
  mpg ~ wt + hp + qsec + disp + drat + gear,
  data = mtcars,
  lambda = lambda_grid
)

ridge_fit$coef
ridge_fit$GCV
best_index <- which.min(ridge_fit$GCV)
lambda_grid[best_index]
coef(ridge_fit)[best_index, ]
```



```
plot(ridge_fit)
select(ridge_fit)
```

Ridge uses an L2 penalty. Coefficients shrink smoothly toward zero but normally do not become exactly zero.

6.3 Ridge and LASSO with glmnet

If the `glmnet` package is installed, use:

```
library(glmnet)

x <- model.matrix(
  mpg ~ wt + hp + qsec + disp + drat + gear,
  data = mtcars
)[, -1]
y <- mtcars$mpg

ridge_glmnet <- glmnet(x, y, alpha = 0)
lasso_glmnet <- glmnet(x, y, alpha = 1)

plot(ridge_glmnet, xvar = "lambda")
plot(lasso_glmnet, xvar = "lambda")

cv_ridge <- cv.glmnet(x, y, alpha = 0, nfolds = 5)
cv_lasso <- cv.glmnet(x, y, alpha = 1, nfolds = 5)

cv_lasso$lambda.min
cv_lasso$lambda.1se
coef(cv_lasso, s = "lambda.min")
predict(cv_lasso, newx = x, s = "lambda.min")
```

LASSO uses an L1 penalty and can set coefficients exactly to zero. The `alpha` argument is 0 for ridge and 1 for LASSO.

6.4 Soft thresholding

```
soft_threshold <- function(z, gamma) {
  sign(z) * max(abs(z) - gamma, 0)
}
soft_threshold(2.5, 1)
soft_threshold(-2.5, 1)
```

Soft thresholding is the basic coordinate-descent update behind a simple LASSO implementation.

6.5 AIC, BIC, and stepwise selection

```
reduced <- lm(mpg ~ wt + hp, data = mtcars)

AIC(reduced, full)
BIC(reduced, full)
anova(reduced, full)

step(full, direction = "forward", trace = 0)
step(full, direction = "backward", trace = 0)
step(full, direction = "both", trace = 0)
```

```
bic_model <- step(
  full,
  direction = "both",
  k = log(nrow(mtcars)),
  trace = 0
)
formula(bic_model)
```

AIC generally uses a lighter complexity penalty. BIC uses $k = \log(n)$ and is usually more parsimonious.

6.6 Cross-validation

```
set.seed(452)
fold <- sample(rep(1:5, length.out = nrow(mtcars)))

cv_mse <- function(formula, data, fold) {
  mean(sapply(sort(unique(fold)), function(k) {
    train <- data[fold != k, ]
    test <- data[fold == k, ]
    fit <- lm(formula, data = train)
    mean((test$mpg -
           predict(fit, newdata = test))^2)
  })))
}

cv_mse(mpg ~ wt, mtcars, fold)
cv_mse(mpg ~ wt + hp + qsec + disp + drat + gear,
       mtcars, fold)
```

Cross-validation estimates performance on observations not used to fit the corresponding model. It is more informative for prediction than training SSE.

6.7 Leave-one-out cross-validation

```
library(boot)

loocv <- cv.glm(
  data = mtcars,
  glmfit = glm(mpg ~ wt + hp, data = mtcars),
  K = nrow(mtcars)
)
loocv$delta
```

7 Topic 4: Logistic regression and classification

7.1 Create a binary response

```
data(mtcars)

cars <- transform(
  mtcars,
```

```

transmission = factor(
  am,
  levels = c(0, 1),
  labels = c("automatic", "manual")
)
)
table(cars$transmission)

```

7.2 Fit a logistic model

```

logit_fit <- glm(
  transmission ~ wt + hp,
  data = cars,
  family = binomial(link = "logit")
)

summary(logit_fit)
coef(logit_fit)
confint.default(logit_fit)

```

The logistic model is

$$\log \left(\frac{\pi(x)}{1 - \pi(x)} \right) = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p,$$

where $\pi(x) = P(Y = 1 \mid x)$.

7.3 Odds ratios

```

odds_ratios <- exp(coef(logit_fit))
odds_ratios

odds_ratio_intervals <- exp(confint.default(logit_fit))
odds_ratio_intervals

exp(10 * coef(logit_fit)["hp"])

```

`exp(beta)` is the multiplicative change in odds for a one-unit increase in a predictor, holding the other predictors fixed.

7.4 Predicted probabilities

```

prob_manual <- predict(logit_fit, type = "response")
linear_predictor <- predict(logit_fit, type = "link")

head(prob_manual)
range(prob_manual)

new_car <- data.frame(wt = 3, hp = 110)
predict(logit_fit, newdata = new_car,
  type = "response")
predict(logit_fit, newdata = new_car,
  type = "link")

```

Use `type = "response"` for probabilities and `type = "link"` for fitted log-odds.

7.5 Classification

```
threshold <- 0.5
predicted_class <- ifelse(
  prob_manual >= threshold,
  "manual",
  "automatic"
)
predicted_class <- factor(
  predicted_class,
  levels = levels(cars$transmission)
)

table(
  observed = cars$transmission,
  predicted = predicted_class
)
mean(predicted_class == cars$transmission)
```

7.6 Confusion-matrix metrics

```
confusion <- table(
  observed = cars$transmission,
  predicted = predicted_class
)

TP <- confusion["manual", "manual"]
TN <- confusion["automatic", "automatic"]
FP <- confusion["automatic", "manual"]
FN <- confusion["manual", "automatic"]

accuracy <- (TP + TN) / sum(confusion)
sensitivity <- TP / (TP + FN)
specificity <- TN / (TN + FP)
precision <- TP / (TP + FP)

c(accuracy = accuracy,
  sensitivity = sensitivity,
  specificity = specificity,
  precision = precision)
```

Sensitivity is the true-positive rate. Specificity is the true-negative rate. The best threshold depends on the relative costs of false positives and false negatives.

7.7 Compare thresholds

```
classify_at <- function(probability, truth, threshold = 0.5) {
  prediction <- factor(
    ifelse(probability >= threshold,
           "manual", "automatic"),
    levels = levels(truth)
  )
  table(observed = truth, predicted = prediction)
}
```

```
classify_at(prob_manual, cars$transmission, 0.3)
classify_at(prob_manual, cars$transmission, 0.5)
classify_at(prob_manual, cars$transmission, 0.7)
```

7.8 Deviance, AIC, and nested logistic models

```
null_fit <- glm(
  transmission ~ 1,
  data = cars,
  family = binomial
)

anova(null_fit, logit_fit, test = "Chisq")
AIC(null_fit, logit_fit)
BIC(null_fit, logit_fit)
deviance(logit_fit)
logLik(logit_fit)
```

7.9 Logistic diagnostics

```
plot(logit_fit)
residuals(logit_fit, type = "deviance")
residuals(logit_fit, type = "pearson")
hatvalues(logit_fit)
cooks.distance(logit_fit)
```

Check for fitted probabilities near zero or one, complete or quasi-complete separation, influential observations, and poor calibration.

8 Training/test validation

```
set.seed(452)
test_index <- sample(
  seq_len(nrow(cars)),
  size = 8
)

train <- cars[-test_index, ]
test <- cars[test_index, ]

train_fit <- glm(
  transmission ~ wt + hp,
  data = train,
  family = binomial
)

test_probability <- predict(
  train_fit,
  newdata = test,
  type = "response"
)

test_prediction <- ifelse(
```

```

test_probability >= 0.5,
"manual",
"automatic"
)

mean(test_prediction == test$transmission)
table(
  observed = test$transmission,
  predicted = test_prediction
)

```

Training accuracy is often optimistic because the model was fitted on those observations. Test-set or cross-validation performance is more informative about generalization.

9 Common mistakes and final checklist

```

# Correct model syntax
fit <- lm(y ~ x1 + x2, data = d)

# Polynomial term
fit2 <- lm(y ~ x + I(x^2), data = d)

# Matrix multiplication
t(X) %*% X

# Logistic probability
predict(logit_fit, type = "response")

# New data must use original predictor names
new_data <- data.frame(x1 = 3, x2 = 110)
predict(fit, newdata = new_data)

```

- Use `~` to define a model formula.
- Use `I(x^2)` or `poly(x, 2)` for polynomial terms.
- Use `%*%`, not `*`, for matrix multiplication.
- Standardize predictors before ridge or LASSO.
- Use `type = "response"` for logistic probabilities.
- Match factor levels and predictor names between training and new data.
- Interpret multiple-regression slopes while holding other predictors fixed.
- Do not use p-values as the only variable-selection criterion.
- Do not delete influential observations automatically.
- Do not extrapolate polynomial models casually.

inspect → fit → diagnose → validate → interpret
